Computer Communications 154 (2020) 197–203

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

ADMM penalized decoding method based on improved penalty function for LDPC codes in the IoTs

Biao Wang, Zhongfei Wang *

School of Mathematics and Information Science, Baoji University of Arts and Sciences, Baoji, Shaanxi 721013, China

ARTICLE INFO

Keywords:
Internet of Things
Alternating direction method of multipliers
Low-density parity-check codes
Penalty function
Decoding speed

ABSTRACT

Low-density parity-check (LDPC) codes is widely used in communication technology which is the key technology of the internet of things (IoTs). By increasing the cost of the pseudocodewords, the alternating direction method of multipliers (ADMM) penalized decoding method improves the decoding performance for LDPC codes at low signal-to-noise ratios. In order to increase the ADMM penalized decoding speed, an improved penalty function, which combines the original l_1 and l_2 penalty functions to form a piecewise function for penalized decoding, is proposed in this letter. The simulation results show that compared with the original ADMM penalized decoding methods for LDPC codes, the improved method obtains better decoding performance and reduces the average decoding time.

1. Introduction

In recent years, the internet of things (IoTs) has drawn significant research attention [1–4]. The IoTs is an important stage of development in the information age [5–8], and it has three layers: application layer, network layer and perception layer. The basic model and hierarchy of IoTs are shown in Figs. 1 and 2, respectively. Communication technology is the key technology of the IoTs, and low-density parity-check (LDPC) codes is widely used in the field of communication [9–13]. The alternating direction method of multipliers (ADMM) is a typical distributed optimization method with good performance and a strict theoretical guarantee [14]. ADMM has been widely used in many fields, such as data mining, compressed sensing, matrix fitting, pattern recognition, image processing and other fields, to solve the optimization problems, all can achieve better application results. Linear programming (LP) decoding is an alternative decoding method for LDPC codes [15,16]. Based on ADMM, Barman et al. designed an efficient LP decoding method [17] for LDPC codes. The method used a projection algorithm represented by a "two-slice" to calculate the Euclidean projection of a vector on the check polytope, and it reduced the decoding complexity of the LP decoding method. Zhang et al. proposed a reduced-complexity algorithm based on the cut search algorithm for the ADMM-based LP decoding method [18]. Recently, a method for simplifying the Euclidean projection through the simplex was designed [19], which also reduced the decoding complexity. A hardware compatible projection algorithm is proposed in [20] by combing the ideas in [18] and [19]. Then, Wei et al. reduced the complexity of the ADMM-based LP decoding method by directly reducing the number of Euclidean projections [21]. Through using lookup tables, Jiao et al. also reduced the complexity of the ADMM-based LP decoding method [22]. To simplify the Euclidean projection, an iterative algorithm, without sorting operation, is proposed for projection onto the check polytope [23].

Although the ADMM-based LP decoding method reduces the decoding complexity of LDPC codes, the simulation results show that the decoding performance is worse than that of the belief propagation (BP) decoding method at low signal-to-noise ratios (SNRs). In order to solve this problem, Liu et al. designed an improved ADMM decoding (called ADMM penalized decoding) method by adding a penalty function to the objective function of the LP decoding problem [24]. The method can improve the decoding performance of LDPC codes by increasing the cost of the pseudocodewords (specifically, by punishing the pseudocodewords) at low SNRs. Jiao et al. proposed an improved ADMM penalized decoding method by setting different parameters for variable nodes with different degrees which could obviously improve the decoding performance [25]. By changing the message passing strategy between variable nodes and check nodes, Debbabi et al. designed two layered scheduling ADMM penalized decoding methods that not only improve the decoding performance but also increase the convergence speed of LDPC codes [26,27]. A node-wise scheduling algorithm also reduced the complexity of ADMM decoding for LDPC codes [28].

In order to further increase the ADMM penalized decoding speed of LDPC codes, this letter proposes an improved penalty function that combines the l_1 penalty function with the l_2 penalty function to form a piecewise function for the penalized decoding. The improved penalty function uses the l_1 penalty function on $x \in (0, \frac{1}{3})$ and $x \in (\frac{2}{3}, 1)$, and the l_2 penalty function on $x \in [\frac{1}{3}, \frac{2}{3}]$. Therefore, it can punish

* Corresponding author.
E-mail addresses: wangbiao401@163.com (B. Wang), loverkind0908@126.com (Z. Wang).

https://doi.org/10.1016/j.comcom.2020.02.063
Received 3 January 2020; Received in revised form 28 January 2020; Accepted 23 February 2020
Available online 26 February 2020
0140-3664/© 2020 Elsevier B.V. All rights reserved.

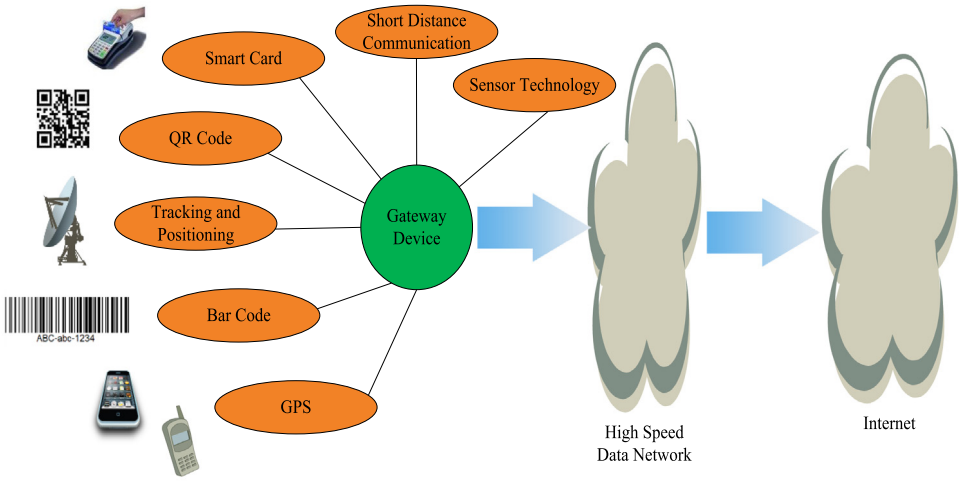


Fig. 1. Basic model of IoTs.

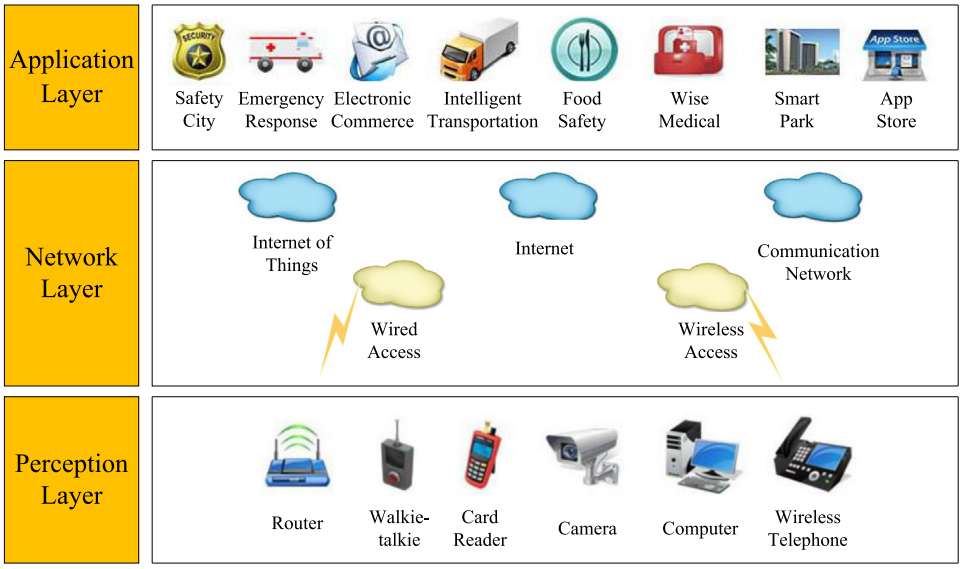


Fig. 2. Hierarchy of IoTs.

the pseudocodewords more flexibly than both the l_1 and l_2 penalty functions. The simulation results show that the ADMM penalized decoding method based on the improved penalty function can obtain better decoding performance and less average decoding time.

The remainder of the letter is organized as follows. In Section 2, we briefly review the ADMM decoding and ADMM penalized decoding for LDPC codes. In Section 3, we design and comment the improved penalty function. In Section 4, we provide the simulation results. Finally, we conclude the letter by Section 5.

2. Preliminaries

If an LDPC code C is described by an $m \times n$ parity-check matrix $\boldsymbol{H}$, then its Tanner graph has n variable nodes denoted by $I = \{1, 2, \ldots, n\}$ and m check nodes denoted by $J = \{1, 2, \ldots, m\}$. The element $\boldsymbol{H}_{ji} = 1$ in $\boldsymbol{H}$ denotes that the check node c_j associates with the variable node v_i. $N_v(i)$ denotes the set of all check nodes which associate with the variable node v_i, $d_{v_i} = |N_v(i)|$ denotes the degree of v_i; similarly, $N_c(j)$ denotes the set of all variable nodes that associate with check node c_j, and $d_{c_j} = |N_c(j)|$ denotes the degree of c_j.

Suppose a codeword $\boldsymbol{x} = \{x_i \in \{0,1\} | i \in I\}$ is transmitted over a memoryless binary-input output-symmetric channel, and a vector $r = \{r_i | i \in I\}$ is received. The vector of log-likelihood ratios (LLRs), $\boldsymbol{\gamma} = \gamma_i | i \in I$, is defined as $\gamma_i = \log\left(\frac{\Pr(r_i|0)}{\Pr(r_i|1)}\right)$, where $\Pr()$ indicates the probability of events in brackets. In order to make the LP decoding problem fit the ADMM template, we introduce the "replica" variables $\boldsymbol{z}_j \in \mathrm{R}^{d_{c_j}}$ $(j \in J)$, and the LP decoding problem is rewritten as [17]

$$\begin{cases} \min & \boldsymbol{\gamma}^T \boldsymbol{x} \\ \text{s.t.} & \mathrm{T}_j \boldsymbol{x} = \boldsymbol{z}_j, \boldsymbol{z}_j \in \mathrm{P}_{d_{c_j}}, \forall j \in J \end{cases} \tag{1}$$

where the check polytope $\mathrm{P}_{d_{c_j}}$ is the convex hull of all even-parity binary vectors of length d_{c_j}; a $d_{c_j} \times n$ binary transfer matrix, T_j, can select the d_{c_j} components of $\boldsymbol{x}$ that participate in the check node c_j.

The augmented Lagrangian of problem (1) is

$$L_\mu(\boldsymbol{x}, \boldsymbol{z}, \boldsymbol{y}) = \boldsymbol{\gamma}^{\mathrm{T}} \boldsymbol{x} + \frac{\mu}{2} \sum_{j \in J} \left\| \mathrm{T}_j \boldsymbol{x} - \boldsymbol{z}_j + \boldsymbol{y}_j \right\|_2^2 - \frac{\mu}{2} \sum_{j \in J} \left\| \boldsymbol{y}_j \right\|_2^2 \tag{2}$$

where $\boldsymbol{y}_j \in \mathrm{R}^{d_{c_j}}$ are the scaled dual variables corresponding to each check node c_j, the constant μ is a penalty parameter and $\mu > 0$.

The ADMM iterations for the problem (1) can be described as

$$\boldsymbol{x}^{k+1} := \underset{\boldsymbol{x} \in \boldsymbol{X}}{\operatorname{argmin}} (\boldsymbol{\gamma}^{\mathrm{T}} \boldsymbol{x} + \frac{\mu}{2} \sum_{j \in J} \left\| \mathrm{T}_j \boldsymbol{x} - \boldsymbol{z}_j^k + \boldsymbol{y}_j^k \right\|_2^2) \tag{3}$$

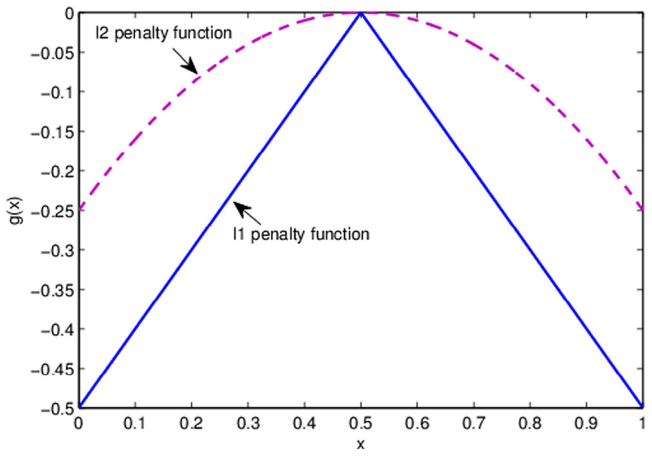


Fig. 3. Images of l_1 and l_2 penalty functions.

$$\boldsymbol{z}_j^{k+1} := \prod_{\mathrm{P}_{d_{c_j}}} (\mathrm{T}_j \boldsymbol{x}^{k+1} + \boldsymbol{y}_j^k) \tag{4}$$

$$\boldsymbol{y}_j^{k+1} := \boldsymbol{y}_j^k + \mathrm{T}_j \boldsymbol{x}^{k+1} - \boldsymbol{z}_j^{k+1} \tag{5}$$

where $\boldsymbol{X} = [0,1]^n$, $\prod_{\mathrm{P}_{d_{c_j}}}(u)$ denotes the Euclidean projection of the vector u onto $\mathrm{P}_{d_{c_j}}$.

For convenience, the update of the vector $\boldsymbol{x}$ in (3) can be transformed in the component-wise fashion:

$$x_i = \Pi_{[0,1]} \left[\frac{1}{d_{v_i}} \Big(\sum_{j \in N_v(i)} \left(\mathrm{z}_j^{(i)} - \mathrm{y}_j^{(i)} \right) - \frac{1}{\mu} \gamma_i \Big) \right] \tag{6}$$

where $\mathrm{z}_j^{(i)}$ and $\mathrm{y}_j^{(i)}$ represent the entries in $\boldsymbol{z}_j$ and $\boldsymbol{y}_j$ that correspond to the variable node v_i, $\Pi_{[0,1]}(u)$ denotes the projection of u to the interval [0, 1]. The operation rule of $\Pi_{[0,1]}(u)$ is

$$\prod_{[0,1]} (u) = \begin{cases} 1, u > 1 \\ u, \ 0 \le u \le 1 \\ 0, u < 0 \, . \end{cases}$$

As the ADMM penalized decoder adds a penalty function in (1), then its objective function is changed to

$$f(\boldsymbol{x}) = \boldsymbol{\gamma}^T \boldsymbol{x} + \alpha \sum_{i \in I} g(x_i) \tag{7}$$

where α is a penalty parameter and $g(x)$ is a penalty function. In [24], two penalty functions are introduced, which are the l_1 penalty function $g_1(x) = -|x - 0.5|$ and the l_2 penalty function $g_2(x) = -(x - 0.5)^2$. The images of l_1 and l_2 penalty functions are shown in Fig. 3.

The penalty function $g(x)$ has the following properties [24]:

(i) $g(x)$ is an increasing function on $(0, \frac{1}{2})$;

(ii) $g(x) = g(1 - x)$ for $x \in [0, 1]$;

(iii) $g(x)$ is differentiable on $(0, \frac{1}{2})$;

(iv) $g(x)$ is such that the $\boldsymbol{x}$-update of ADMM is well defined.

Property (i) penalizes fractional solutions. Property (ii) ensures that the decoding error is independent with respect to the transmitted codeword. And we can determine the ADMM updates with explicit forms through properties (iii) and (iv).

In the ADMM penalized decoder, the $\boldsymbol{z}_j$- and $\boldsymbol{y}_j$-updates do not change, and the $\boldsymbol{x}$-update is changed into

$$x_i = \Pi_{[0,1]} \left[\frac{1}{d_{v_i}} \Big(\sum_{j \in N_v(i)} \left(\mathrm{z}_j^{(i)} - \mathrm{y}_j^{(i)} \right) - \frac{1}{\mu} (\nabla_x f)_i \Big) \right] \tag{8}$$

where $\nabla_x f$ denotes the differential of f with respect to x.

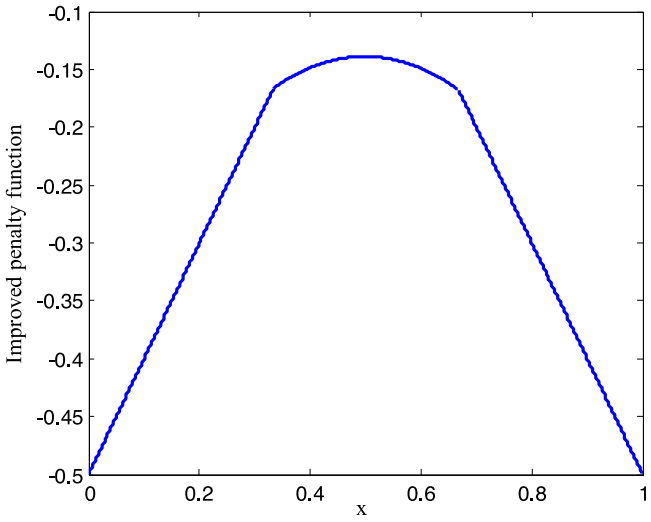


Fig. 4. Image of improved penalty function.

3. Improved penalty function

3.1. Design of improved penalty function

In [24], we find that the decoding performance of the ADMM penalized decoding method based on the l_1 penalty function is approximately the same as that of the method based on the l_2 penalty function, but their average number of iterations are different. Under the same conditions, the average number of iterations of the ADMM penalized decoding method based on the l_2 penalty function is lower than that of the method based on the l_1 penalty function. It shows that the convergence rate of the ADMM penalized decoding method based on the l_1 penalty function is slower than that of the method based on the l_2 penalty function. When $x \in (0, \frac{1}{2})$, the slope of the l_1 penalty function is 1 and the slope of the l_2 penalty function is decreased from 1 to 0, it is clear that the change in the slope of the two penalty functions is different. Therefore, we consider that the convergence rate of the ADMM penalized decoding method may be related to the slope of the penalty function.

Based on this consideration, in order to punish the pseudocodewords more quickly (namely, to increase the ADMM penalized decoding speed of the LDPC codes), this letter proposes an improved penalty function that combines the l_1 penalty function with the l_2 penalty function to form a piecewise function. Thus, the improved penalty function can punish the pseudocodewords quickly near $x = 0$ and $x = 1$, and punish the pseudocodewords smoothly near $x = 0.5$.

The proposed improved penalty function is designed as

$$g(x) = \begin{cases} x - 0.5, & 0 \le x < \frac{1}{3} \\ -(x - 0.5)^2 - \frac{5}{36}, & \frac{1}{3} \le x \le \frac{2}{3} \\ -x + 0.5, & \frac{2}{3} < x \le 1 \end{cases} \tag{9}$$

and the image of improved penalty function is shown in Fig. 4.

Let $t_i = \sum_{j \in N_v(i)} \left(\mathrm{z}_j^{(i)} - \mathrm{y}_j^{(i)} \right) - \frac{\gamma_i}{\mu}$. Each component update of $\boldsymbol{x}$ in the ADMM penalized decoding method based on the improved penalty

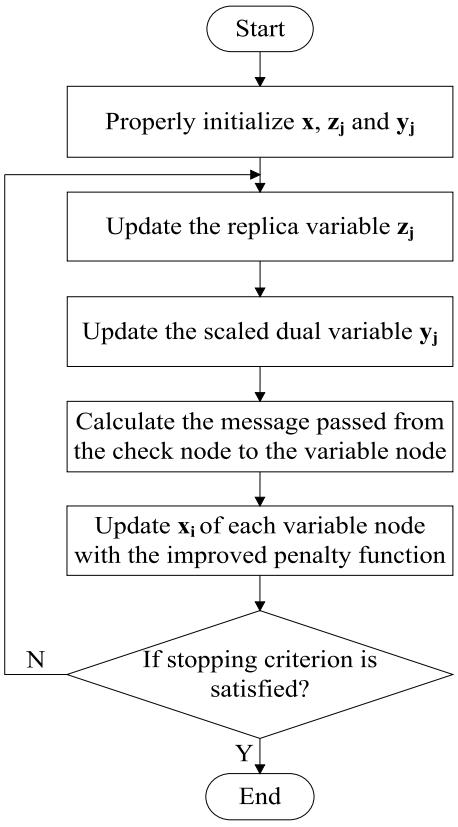


Fig. 5. Algorithm flow of ADMM penalized decoding method with improved penalty function.

function is as follows:

$$x_i = \begin{cases} \Pi_{[0,1]}\left[\frac{1}{d_{v_i}}(t_i - \frac{\alpha}{\mu})\right], t_i < \frac{1}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu} \\ \Pi_{[0,1]}\left[\frac{1}{d_{v_i} - \frac{2\alpha}{\mu}}(t_i - \frac{\alpha}{\mu})\right], \frac{1}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu} \le t_i \le \frac{2}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu} \\ \Pi_{[0,1]}\left[\frac{1}{d_{v_i}}(t_i + \frac{\alpha}{\mu})\right], t_i > \frac{2}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu} \end{cases} \tag{10}$$

where α is the penalty parameter for the improved penalty function, μ is the penalty parameter for the augmented Lagrangian of problem (1).

It is should be noted that the x of the proposed improved penalty function (9) is the x_i of the updating rules (10) which linear corresponds to t_i in the updating rules (10). And the specific derivation of the updating rules (10) is: when $x = \frac{1}{3}$, we can solve $t_i = \frac{1}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu}$ according to the second piece of the penalty function (9); similarly, $t_i = \frac{2}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu}$ can be solved out when $x = \frac{2}{3}$. Therefore, when $\frac{1}{3} \le x \le \frac{2}{3}$, corresponding $\frac{1}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu} \le t_i \le \frac{2}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu}$, and we update the rules (10) in accordance with the second piece of the penalty function (9); when $x < \frac{1}{3}$, corresponding $t_i < \frac{1}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu}$, we update the rules (10) in accordance with the first piece of the penalty function (9); when $x > \frac{2}{3}$, corresponding $t_i > \frac{2}{3}(d_{v_i} - \frac{2\alpha}{\mu}) + \frac{\alpha}{\mu}$, we update the rules (10) in accordance with the third piece of the penalty function (9).

The algorithm flow of ADMM penalized decoding method based on the proposed improved penalty function is shown in Fig. 5.

3.2. Comments on the improved penalty function

Comment 1: If a penalty function has the property (i) and (ii), then the Ideal ML Test, Weak ML Test and All-Zero Assumption hold for the ADMM penalized decoding method [24]. It is clear that the improved penalty function in the letter has the property (i) and (ii), so the Ideal ML Test, Weak ML Test and All-Zero Assumption also hold for the ADMM penalized decoding method based on the improved penalty function. Of course, the improved penalty function does not have the property (iii), but the $\boldsymbol{x}$-update of ADMM is still well defined with it, so it can be used as a penalty function in the ADMM penalized decoder.

It should be noted that the ML certificate property of LP decoder will be lost if the ADMM penalized decoder is used. So does the ADMM decoder with the improved penalty function. However, in [24], Liu et al. showed that the ML certificate property of ADMM penalized decoder can be tested by performing one more iteration of the ADMM decoding.

Comment 2: Compared with the original ADMM penalized decoding method, the ADMM penalized decoding method based on the improved penalty function only changes the penalty function of the Algorithm 1 in [24], while other parts of the algorithm remain unchanged. Thus, the decoding complexity of the proposed method in this letter is approximately the same as that of the original ADMM penalized decoding method.

4. Simulation results

In this section, two LDPC codes are used to compare the decoding performance, the average number of iterations, and the average decoding time of the ADMM penalized decoding methods with the improved penalty function, l_1 and l_2 penalty functions. The two LDPC codes, which are (576, 288) rate-0.5 (called C_1) and (1152, 288) rate-0.75 (called C_2), are chosen from IEEE 802.16e standard [29]. The code length and information bit length of C_1 are 576 and 288, and that of C_2 are 1152 and 864, respectively. For C_1 and C_2, the parameters α and μ of the ADMM penalized method with the l_1, l_2 and improved penalty functions are optimized according to the method in [17]. This letter uses the $\boldsymbol{H}\boldsymbol{x}^T = \boldsymbol{0}$ early termination (ET) technology in all experiments, and the maximum number of iterations, the error tolerance and the over-relaxation parameter [30] are set to 1000, 10^{-5} and 1.9, respectively.

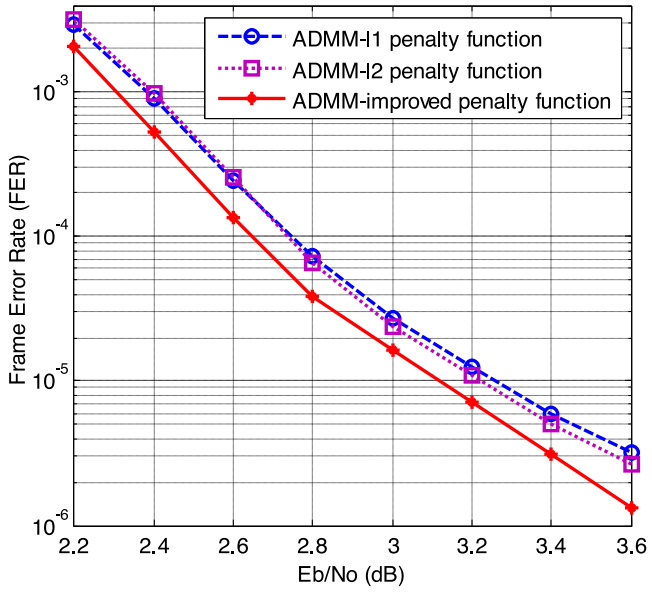
4.1. Decoding performance

The comparison of the frame error rate (FER) performance of the two LDPC codes is shown in Fig. 6. And the FER performance is obtained by generating 200 error frames.

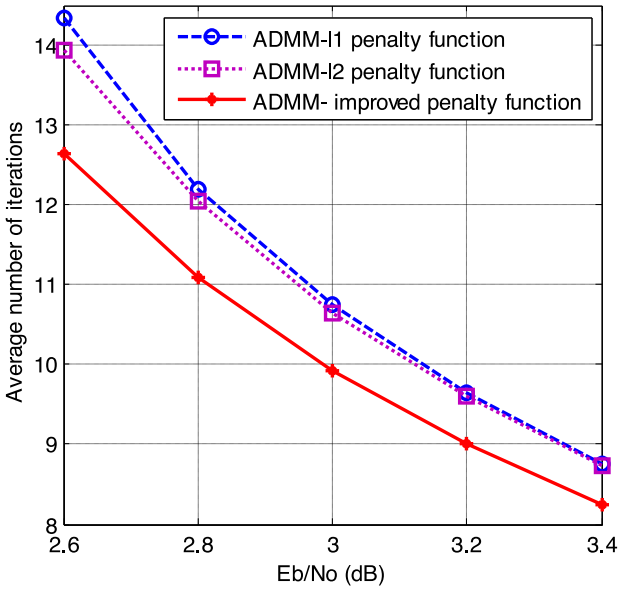
We can see that the FER performance of the ADMM penalized decoding method with the improved penalty function is better than that of the ADMM penalized decoding methods with the l_1 and l_2 penalty functions. For C_1, when FER = 10^{-5}, the performance gains of the improved penalty function are about 0.12 dB and 0.1 dB, compared with the l_1 and l_2 penalty functions, respectively. Similarly, for C_2, when FER = 10^{-5}, the performance gains are about 0.15 dB and 0.05 dB.

4.2. Average number of iterations

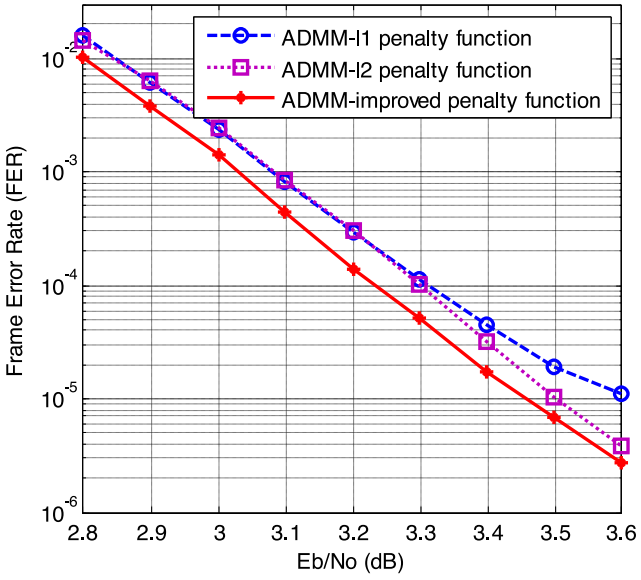
Fig. 7 shows the comparison of the average number of iterations of the two LDPC codes. It can be seen that the average number of iterations of the ADMM penalized decoding method with the improved penalty function is lower than that of the ADMM penalized decoding methods with the l_1 and l_2 penalty functions. For C_1, when SNR = 2.8 dB, the average number of iterations of the ADMM penalized decoding method with the improved penalty function are reduced by approximately 9.13% and 8.06%, compared with the l_1 and l_2 penalty



(a) FER performance of C_1

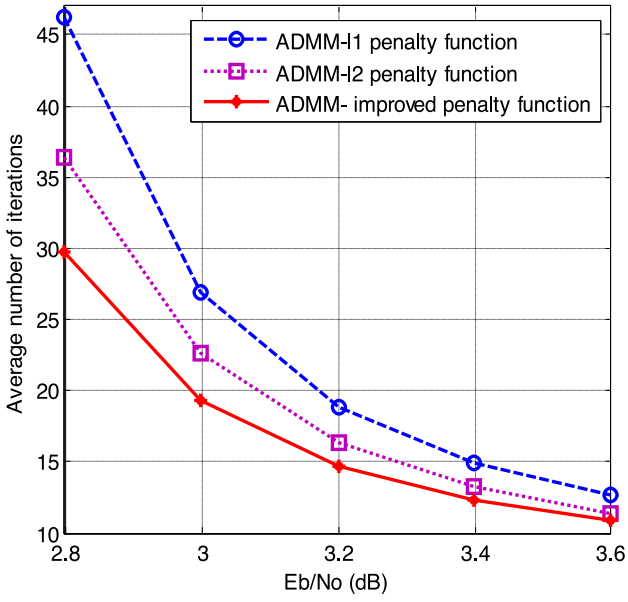


(a) Average number of iterations of C_1



(b) FER performance of C_2

Fig. 6. FER performance of C_1 and C_2.



(b) Average number of iterations of C_2

Fig. 7. Average number of iterations of C_1 and C_2.

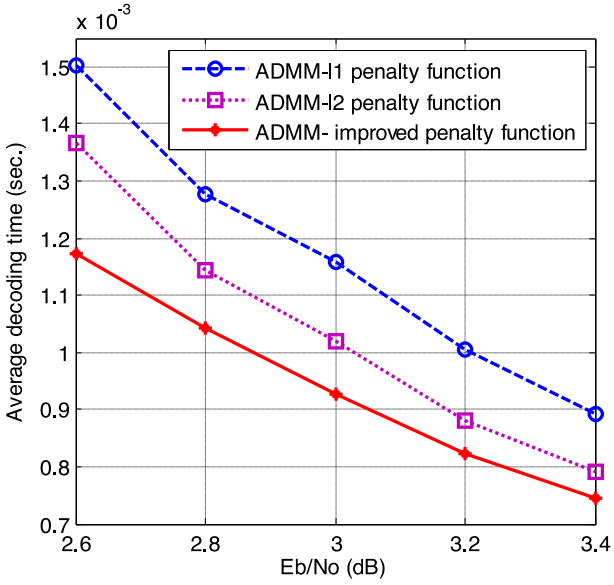
functions, respectively. Similarly, for C_2, when SNR = 2.8 dB, the average number of iterations of the ADMM penalized decoding method with the improved penalty function are reduced by approximately 35.59% and 18.07%.

4.3. Average decoding time

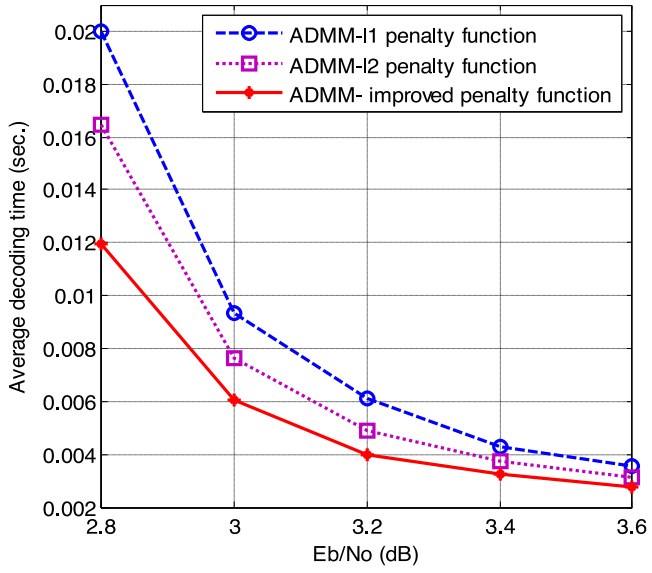
The comparison of the average decoding time of the two LDPC codes is shown in Fig. 8. We can see that the average decoding time of the ADMM penalized decoding method with the improved penalty function is less than that of the ADMM penalized decoding methods with the l_1 and l_2 penalty functions. For C_1, when SNR = 2.8 dB, the average decoding time of the ADMM penalized decoding method with the improved penalty function are decreased by approximately 18.18% and 8.66%, compared with the l_1 and l_2 penalty functions, respectively. Similarly, for C_2, when SNR = 2.8 dB, the average decoding time of the ADMM penalized decoding method with the improved penalty function are decreased by approximately 40.31% and 27.36%.

5. Conclusion

In order to increase the ADMM penalized decoding speed, this letter designs an improved penalty function through combining the l_1 penalty function with the l_2 penalty function. The improved penalty function can punish the pseudocodewords quickly near $x = 0$ and $x = 1$, and punish the pseudocodewords smoothly near $x = 0.5$. Two LDPC codes in the IEEE 802.16e standard are simulated to show that the ADMM penalized decoding method with the improved penalty function can improve the decoding performance and reduce the average number of iterations and the average decoding time. In the future, we will study



(a) Average decoding time of C_1



(b) Average decoding time of C_2

Fig. 8. Average decoding time of C_1 and C_2.

the application of the improved penalty function for hardware designs for real-time LDPC decoding in the IoTs.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Biao Wang: Formal analysis. **Zhongfei Wang:** Data curation.

Acknowledgments

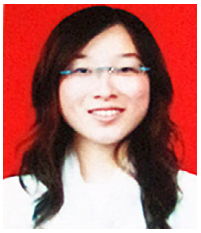
This work is supported by the National Nature Science Foundation of China (No. 61602010), the Science and Technology Plan Project in Baoji of China (No. 2018JH-18).

References

[1] M. Zheng, S. Chen, W. Liang, M. Song, NSAC: a novel clustering protocol in cognitive radio sensor networks for internet of things, IEEE Internet Things J. 6 (3) (2019) 5864–5865.
[2] E. Giaccardi, Things making things: designing the internet of reinvented things, IEEE Pervasive Comput. 17 (3) (2018) 70–72.
[3] M. Shahzad, M.P. Singh, Continuous authentication and authorization for the internet of things, IEEE Internet Comput. 21 (2) (2017) 86–90.
[4] J.M. Chen, K. Hu, Q. Wang, Y.Y. Sun, Z.G. Shi, S.B. He, Narrowband internet of things: implementations and applications, IEEE Internet Things J. 4 (6) (2017) 2309–2314.
[5] M.P.K. Reddy, M.R. Babu, Implementing self adaptiveness in whale optimization for cluster head section in internet of things, Cluster Comput. 22 (10) (2019) 1361–1372.
[6] A. Apse, A simple guide to low-power wireless technologies: balancing the tradeoffs for the internet of things and medical applications, IEEE Solid-State Circuits Mag. 10 (4) (2018) 16–23.
[7] L. Davoli, L. Belli, A. Cilfone, G. Ferrari, From micro to macro iot: challenges and solutions in the integration of IEEE 802.15.4/802.11 and sub-GHz technologies, IEEE Internet Things J. 5 (2) (2018) 784–793.
[8] A. Samuel, C. Sipes, Making internet of things real, IEEE Internet Things Mag. 2 (1) (2019) 10–12.
[9] H.Y. Kwak, J.S. No, H. Park, Design of irregular SC-LDPC codes with non-uniform degree distributions by linear programming, IEEE Trans. Commun. 67 (4) (2019) 2632–2646.
[10] M.H. Tadayon, A. Tasdighi, M. Battaglioni, M. Baldi, F. Chiaraluce, Efficient search of compact QC-LDPC and SC-LDPC convolutional codes with large girth, IEEE Commun. Lett. 22 (6) (2018) 1156–1159.
[11] M.R. Sadeghi, Optimal search for girth-8 quasi cyclic and spatially coupled multiple-edge LDPC codes, IEEE Commun. Lett. 23 (9) (2019) 1466–1469.
[12] A. Dehghan, A.H. Banihashemi, On the tanner graph cycle distribution of random LDPC, random protograph-based LDPC, and random quasi-cyclic LDPC code ensembles, IEEE Trans. Inf. Theory (2018).
[13] I. Ali, H. Lee, A. Hussain, S.H. Kim, Protograph-based folded spatially coupled LDPC codes for burst erasure channels, IEEE Wirel. Commun. Lett. 8 (2) (2019) 516–519.
[14] X. Lai, J. Cao, Z. Lin, A novel relaxed ADMM with highly parallel implementation for extreme learning machine, in: Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS), Florence, 27-30 2018, 2018, pp. 1-5.
[15] J. Bai, Y. Wang, F.C.M. Lau, Minimum-polytope-based linear programming decoder for LDPC codes via ADMM approach, IEEE Wirel. Commun. Lett. 8 (4) (2019) 1032–1035.
[16] Z. Liang, X. Chen, X. Sun, L. Zhai, 2019. A reduced-complexity ADMM based decoding algorithm for LDPC codes, in: Proceedings of International Conference on Wireless Communications and Signal Processing (WCSP), Xi'an, 23-25 2019, pp. 1-6.
[17] S. Barman, X. Liu, S.C. Draper, B. Recht, Decomposition methods for large scale LP decoding, IEEE Trans. Inform. Theory 59 (12) (2013) 7870–7886.
[18] X. Zhang, P.H. Siegel, Efficient iterative LP decoding of LDPC codes with alternating direction method of multipliers, in: Proceedings of IEEE International Symposium on Information Theory, Istanbul, 7-12 2013, 2013, pp. 1501-1505.
[19] G. Zhang, R. Heusdens, W.B. Kleijn, Large scale LP decoding with low complexity, IEEE Commun. Lett. 17 (11) (2013) 2152–2155.
[20] M. Wasson, S.C. Draper, Hardware based projection onto the parity polytope and probability simplex, in: Proceedings of Asilomar Conference on Signals, Systems and Computers, Pacific Grove, CA, 8-11 2015, 2015, pp. 1015-1020.
[21] H. Wei, X. Jiao, J. Mu, Reduced-complexity linear programming decoding based on ADMM for LDPC codes, IEEE Commun. Lett. 19 (6) (2015) 909–912.
[22] X. Jiao, J. Mu, Y. He, C. Chen, Efficient ADMM decoding of LDPC codes using lookup tables, IEEE Trans. Commun. 65 (4) (2017) 1425–1436.
[23] H. Wei, A.H. Banihashemi, An iterative check polytope projection algorithm for ADMM-based LP decoding of LDPC codes, IEEE Commun. Lett. 22 (1) (2018) 29–32.
[24] X. Liu, S.C. Draper, The ADMM penalized decoder for LDPC codes, IEEE Trans. Inform. Theory 62 (6) (2016) 2966–2984.
[25] X. Jiao, H. Wei, J. Mu, C. Chen, Improved ADMM penalized decoder for irregular low-density parity-check codes, IEEE Commun. Lett. 19 (6) (2015) 913–916.
[26] I. Debbabi, B.L. Gal, N. Khouja, F. Tlili, C. Jego, Fast converging ADMM-penalized algorithm for LDPC decoding, IEEE Commun. Lett. 20 (4) (2016) 644–647.
[27] I. Debbabi, B.L. Gal, N. Khouja, F. Tlili, C. Jego, Comparison of different schedulings for the ADMM based LDPC decoding, in: Proceedings of International Symposium on Turbo Codes and Iterative Information Processing (ISTC), Brest, 5-9 2016, 2016, pp. 51-55.
[28] X. Jiao, J. Mu, H. Wei, Reduced complexity node-wise scheduling of ADMM decoding for LDPC codes, IEEE Commun. Lett. 21 (3) (2017) 472–475.
[29] LDPC Coding for OFDMA PHY, IEEE Standard C802.16e–05/0066r3, Jan. 2005.
[30] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, Distributed optimization and statistical learning via the alternating direction method of multipliers, Found. Trends Mach. Learn. 3 (1) (2011) 1–122.



Biao Wang received the B.Sc. degree in information and computing science form Shaanxi Normal University in 2005, the M.Sc. degree in computer application technology from Shaanxi University of Science and Technology in 2011, and the Ph.D. degree in computer software and theory form Xidian University in 2018. He is currently a lecturer with the School of Mathematics and Information Science, Baoji University of Arts and Sciences. His major research interests include error-correcting coding and information theory. wangbiao401@163.com



Zhongfei Wang received the B.Sc. degree in information and computing science form Shaanxi Normal University in 2005, and the M.Sc. degree in computer application technology from Shaanxi University of Science and Technology in 2010. She is currently a lecturer with the School of Mathematics and Information Science, Baoji University of Arts and Sciences. Her major research interests include error-correcting coding, graphics and image processing. loverkind0908@126.com